

Software Requirements Specification: The Versor Core Mathematical Engine (Project L.O.V.E.)

1. Executive Summary and Architectural Vision

1.1 The Paradigm Shift in Affective Computing

The domain of Affective Computing has long been constrained by two-dimensional models that reduce the complexity of the human experience to static Cartesian coordinates. Traditional frameworks, most notably the Valence-Arousal-Dominance (VAD) model, have served as the industry standard for sentiment analysis and emotion recognition. These models typically plot emotional states as fixed points on a grid: "Happiness" is high valence and high arousal; "Sadness" is low valence and low arousal. While computationally convenient for basic classification tasks, this approach fails to capture the dynamic, relational, and transformative nature of complex human experiences.

Project L.O.V.E. (Listener - Observer - Versor - Experience) represents a fundamental paradigm shift in this field. It proposes a system that maps human emotion not as fixed points, but as dynamic orientations within a three-dimensional conceptual space. The central thesis of Project L.O.V.E. is that emotions are not destinations (points) but vectors of orientation (directions). A transition from one emotional state to another—for instance, the shift from "Anger" to "Grief"—is not a linear translation across a flat plane but a complex rotation involving changes in strictly defined dimensional axes.

This project synthesizes the rigorous qualitative research of Dr. Brené Brown's *Atlas of the Heart*, which identifies 87 distinct emotions and experiences, with the non-commutative algebraic framework of quaternions. By doing so, the system seeks to quantify the "work" required to shift between emotional states, offering a digital representation of emotional elasticity, resistance, and momentum. The architecture is designed to capture, process, and visualize these transitions in real-time, providing users with a "Soul Sphere"—a biometric visualization that does not merely track mood but visualizes the trajectory of the human spirit.

1.2 The Role of the Versor

The L.O.V.E. Stack comprises four distinct modules:

1. **The Listener:** A multi-modal ingestion engine responsible for transcription and semantic normalization.
2. **The Observer:** A contextual state management system utilizing vector storage.
3. **The Versor:** The pure mathematical processing engine.
4. **The Experience:** A high-fidelity 3D visualization interface.

This document serves as the exhaustive Software Requirements Specification (SRS) for **The**

Versor. The Versor is the mathematical heart of the system. It is a stateless microservice dedicated to the geometric topology of emotion. Unlike the Listener, which deals with the ambiguity of language, or the Observer, which handles the persistence of history, the Versor deals in absolute mathematical truths. It is responsible for mapping the normalized scalar outputs of the Listener into the complex 4D hyperspace of quaternions, calculating the geodesic paths between states, and quantifying the "Emotional Work" involved in every human transition.

The Versor must solve specific computational challenges inherent to this novel approach. It must address the limitations of Euler angles—specifically the phenomenon of Gimbal Lock, which serves as a potent computational metaphor for emotional trauma. It must implement Spherical Linear Interpolation (SLERP) to ensure that the visualization of emotional change maintains constant angular velocity, representing the fluidity of human feeling rather than the robotic, linear cuts of traditional interpolation methods. Furthermore, it must execute these calculations with sufficient speed (sub-50ms latency) to drive a real-time conversational interface, necessitating a highly optimized implementation using the Python scientific computing stack.¹

1.3 Document Scope and Intended Audience

This SRS is written for senior software engineers, algorithm specialists, and AI coding agents (such as Claude) who will be tasked with implementing the Versor module. It assumes a deep familiarity with software architecture, API design, and linear algebra. The document provides a comprehensive blueprint, detailing the theoretical underpinnings of the VAC (Valence, Arousal, Connection) model, the specific quaternion operations required, the data structures for API communication, and the testing strategies necessary to validate the system's mathematical integrity.

The requirements detailed herein are binding. They define the strict mathematical and architectural constraints required to ensure that the digital representation of human emotion remains mathematically robust and psychologically valid.

2. Theoretical Framework: The Topology of the Soul

2.1 Limitations of the VAD Model

Traditional affective computing relies heavily on the VAD model. While effective for basic sentiment analysis, this model fails to capture the nuance of complex human experiences. Specifically, the dimension of "Dominance"—often defined as the degree of control one feels over a situation—is insufficient for mapping the relational depth of emotions described by Dr. Brené Brown. In Brown's framework, the primary driver of the human experience is not control, but connection.

The VAD model creates a "flatness" in emotional representation. For example, "Fear" and "Anger" are both high-arousal, negative-valence states. The VAD model distinguishes them via Dominance (Anger is high dominance; Fear is low dominance). However, this fails to capture the *relational* vector. Anger is often a shield or a pushing away (disconnection), while

Fear can be a cry for help (attempted connection) or a withdrawal (disconnection). The VAD model struggles to map these relational dynamics accurately because it lacks a dedicated axis for interpersonal alignment.

2.2 The VAC Dimensional Model

Project L.O.V.E. replaces the "Dominance" axis with "Connection" (\$z\$-axis). This creates a new dimensional space: the **VAC Model** (Valence, Arousal, Connection). This shift allows the system to distinguish between emotions that might have similar Valence and Arousal but opposite relational vectors.

Axis	Symbol	Range	Definition	Theoretical Implication
Valence	\$x\$	\$[-1.0, +1.0]\$	The intrinsic attractiveness (positive) or aversiveness (negative) of an emotion.	Represents the "quality" of the feeling. A shift from Sadness to Joy is a traversal along the \$x\$-axis.
Arousal	\$y\$	\$[-1.0, +1.0]\$	The physiological state of being awoken or stimulated to action.	Represents the "energy" of the feeling. High arousal (Panic) vs. Low arousal (Boredom) defines the intensity of the vector's magnitude.
Connection	\$z\$	\$[-1.0, +1.0]\$	The degree of authentic alignment with oneself or others.	The novel contribution of Project L.O.V.E. It maps the relational stance. Positive \$z\$ implies vulnerability and belonging; Negative \$z\$ implies armoring, shame, or severance.

The introduction of the Connection axis resolves the ambiguity between states like **Pity** and

Compassion. Both are responses to suffering (likely negative valence, moderate arousal). However:

- **Pity** is defined by separation: "I am over here, you are over there." It creates a vertical hierarchy and distance. In the VAC model, this maps to a **Negative Connection (\$z\$)** vector.
 - Compassion is defined by shared humanity: "I am with you." It dissolves hierarchy. In the VAC model, this maps to a Positive Connection (\$z\$) vector.
- The Versor's geometric engine is the only mechanism capable of visualizing the "rotation" required to move from Pity to Compassion—a movement that requires traversing the \$z\$-axis, representing significant "Emotional Work".

2.3 The 13 Categories of Emotion: A Vector Analysis

To build the Versor, we must understand the input data. The Observer module is founded on the "Atlas," a comprehensive database of 87 emotions organized into 13 categories. The Versor must be capable of generating unique quaternion signatures for representative states within these categories.

1. Places We Go When Things Are Uncertain or Too Much

- *Emotions:* Stress, Overwhelm, Anxiety, Worry, Avoidance, Excitement, Dread, Fear, Vulnerability.
- *Vector Characteristics:* Primarily characterized by **High Arousal (\$y > 0.8\$)**.
- *Versor Challenge:* "Overwhelm" represents a saturation point. Computationally, this models a high-velocity rotation that exceeds the user's "Elasticity" metric. The Versor must detect when the rate of change ($\Delta q / \Delta t$) creates a "blur," signaling a state of "flooding" where the system (or user) cannot process new inputs effectively.

2. Places We Go When We Compare

- *Emotions:* Comparison, Admiration, Reverence, Envy, Jealousy, Resentment, Schadenfreude, Freudenfreude.
- *Vector Characteristics:* These states often involve **Negative Valence** combined with **Blocked Connection**. "Resentment" is particularly complex; it is part of this cluster rather than simple anger. In quaternion space, Resentment implies a rigid orientation—a "locking" of the vector that resists rotation despite external inputs.
- *Versor Challenge:* Distinguishing "Schadenfreude" (Joy at others' pain). This would map to **Positive Valence (\$+x\$)** but **Negative Connection (\$-z\$)**, a perverse vector alignment that the system must validly represent.

3. Places We Go When Things Don't Go As Planned

- *Emotions:* Boredom, Disappointment, Expectations, Regret, Discouragement, Resignation, Frustration.
- *Vector Characteristics:* These states represent a "delta" or error function between an expected trajectory (q_{target}) and the actual trajectory (q_{current}).
- *Versor Challenge:* "Frustration" implies **High Arousal** (energy) without movement—a vector "spinning in place" or vibrating. "Resignation" implies a cessation of rotational momentum—a velocity of zero despite a negative Valence state.

4. Places We Go When It's Beyond Us

- *Emotions*: Awe, Wonder, Confusion, Curiosity, Interest, Surprise.
- *Vector Characteristics*: Expansive states. "Awe" and "Wonder" facilitate a transcendence of the self, mapped as **High Connection (\$+z\$)** and variable Valence.
- *Versor Challenge*: "Confusion" represents high entropy or low vector confidence. The Versor must handle inputs where the magnitude $\|\vec{v}\|$ might fluctuate wildly, creating a "jitter" in the quaternion interpolation.

5. Places We Go When Things Aren't What They Seem

- *Emotions*: Amusement, Bittersweetness, Nostalgia, Cognitive Dissonance, Paradox, Irony, Sarcasm.
- *Vector Characteristics*: Complex quaternions where positive and negative Valence coexist.
- *Versor Challenge*: "Bittersweetness" challenges simple scalar representations. The Versor handles this by mapping the *resultant* vector. If Positive and Negative valence cancel out, the vector might collapse to neutral. However, usually, one dominates. Alternatively, "Cognitive Dissonance" is modeled as a state of high internal friction, represented as a high-frequency vibration in the Experience module.

6. Places We Go When We're Hurting

- *Emotions*: Anguish, Hopelessness, Despair, Sadness, Grief.
- *Vector Characteristics*: Generally **Low Arousal, Negative Valence**.
- *Versor Challenge*: "Anguish" is described as physical pain, suggesting a haptic feedback pattern of heavy, slow throbbing. "Hopelessness" is the absence of a future trajectory—a quaternion with no forward vector.

7. Places We Go With Others

- *Emotions*: Compassion, Pity, Empathy, Sympathy, Boundaries, Comparative Suffering.
- *Vector Characteristics*: This cluster defines the **Positive \$z\$-axis (Connection)**.
- *Versor Challenge*: The Versor must distinguish subtle angular differences. Pity might have a similar Valence to Compassion but an orthogonal Connection vector. The "Work" calculation (ϕ) between these two must be significant.

8. Places We Go When We Fall Short

- *Emotions*: Shame, Self-Compassion, Perfectionism, Guilt, Humiliation, Embarrassment.
- *Vector Characteristics*: "Shame" is the intensely painful feeling of being flawed and unworthy of connection (**Negative \$z\$-axis**).
- *Versor Challenge*: "Perfectionism" is a defensive shield—a rigid orientation that resists any modification or rotation. The Versor models this as a state with extremely low "Elasticity."

9. Places We Go When We Search for Connection

- *Emotions*: Belonging, Fitting In, Connection, Disconnection, Insecurity, Invisibility, Loneliness.
- *Vector Characteristics*: "Belonging" vs. "Fitting In" is the critical distinction. "Fitting In" requires changing one's orientation to match the environment (external alignment), whereas "Belonging" is remaining true to one's internal orientation (internal alignment).

10. Places We Go When the Heart Is Open

- *Emotions*: Love, Lovelessness, Heartbreak, Trust, Self-Trust, Betrayal, Defensiveness,

Flooding, Hurt.

- *Vector Characteristics*: "Flooding" is a rapid, overwhelming input that shuts down processing.
- *Versor Challenge*: Computationally modeled as a system interrupt or a "freeze" in the state trajectory where input processing is throttled.

11. Places We Go When Life Is Good

- *Emotions*: Joy, Happiness, Calm, Contentment, Gratitude, Foreboding Joy, Relief, Tranquility.
- *Vector Characteristics*: "Foreboding Joy" is the fear that joy will be taken away—a state of **High Valence** tainted by **High Anxiety (Arousal)**. This creates a trembling or unstable rotational axis.

12. Places We Go When We Feel Wronged

- *Emotions*: Anger, Contempt, Disgust, Dehumanization, Hate, Self-Righteousness.
- *Vector Characteristics*: "Anger" is an action emotion (**High Arousal**). "Dehumanization" is the complete severing of the Connection axis (**$z = -1.0$**).

13. Places We Go to Self-Assess

- *Emotions*: Pride, Hubris, Humility.
- *Vector Characteristics*: "Hubris" is inflated, unconnected pride (**Positive Valence, Negative Connection**). "Humility" is "grounded confidence," suggesting a stable center of gravity ($z \approx 0.5$).

2.4 Why Quaternions? The Mathematical Necessity

The representation of 3D orientation is mathematically non-trivial. The most common method, Euler angles (pitch, yaw, roll), suffers from a critical flaw known as **Gimbal Lock**. This occurs when two axes align, resulting in the loss of a degree of freedom.

In the context of emotional mapping, Gimbal Lock is a potent metaphor for emotional "stuckness"—the state of trauma or flooding where a person loses the ability to pivot or gain perspective. If the Versor were to use Euler angles, a user in a specific state of "Rage" (e.g., Pitch 90 degrees) might mathematically lose the ability to rotate toward "Compassion" without passing through a singularity. This is unacceptable for a system designed to model fluid human experience.

Quaternions avoid this singularity entirely. A quaternion is a hyper-complex number system that allows for the smooth interpolation of rotation across the surface of a 4-dimensional hypersphere. This property makes them ideal for calculating the "shortest path" between two emotional states, which is rarely a straight line in Cartesian space but an arc in conceptual space.

3. Mathematical Engine: The Versor Logic

3.1 Quaternion Fundamentals

A quaternion q is defined as:

$$q = w + xi + yj + zk$$

where $w, x, y, z \in \mathbb{R}$ and i, j, k are imaginary units satisfying:

$$i^2 = j^2 = k^2 = ijk = -1$$

In the L.O.V.E. system, we utilize **Unit Quaternions** (Versors), where the norm $|q| = 1$.

- **The Vector Part** $(xi + yj + zk)$ encodes the **axis of rotation**, derived from the normalized VAC inputs (Valence, Arousal, Connection).
- **The Scalar Part** (w) encodes the **magnitude of the rotation** (related to the angle θ), representing the intensity of the emotion relative to a neutral baseline.

3.2 State Construction Algorithm

The Versor's primary function is converting the normalized scalar inputs from the Listener $(\vec{v} = [v_x, v_y, v_z])$ into a quaternion q_{current} .

Step 1: Input Vector Validation

The input vector $\vec{v}$ consists of raw outputs from semantic analysis.

$$\vec{v} = [v_x, v_y, v_z]$$

Constraints: Each component must be in range $[-1.0, 1.0]$.

Step 2: Magnitude Calculation (Intensity)

Calculate the Euclidean norm to determine the total intensity of the emotional state.

$$|\vec{v}| = \sqrt{v_x^2 + v_y^2 + v_z^2}$$

- **Edge Case:** If $|\vec{v}| \approx 0$ (below a threshold $\epsilon = 1e^{-6}$), the user is in a "Neutral" state. The system returns the Identity Quaternion: $q_{\text{identity}} = 1$ (scalar first).

Step 3: Axis Normalization (Direction)

If $|\vec{v}| > 0$, the rotation axis $\hat{u}$ is determined by normalizing the input vector.

$$\hat{u} = \frac{\vec{v}}{|\vec{v}|} = [u_x, u_y, u_z]$$

This unit vector defines the "quality" of the emotion.

Step 4: Angle Determination (Displacement)

The angle of rotation θ from the neutral identity state is derived from the magnitude.

We map the intensity range $[0.0, 1.0]$ to an angular range $[0, \pi]$ radians (0° to 180°).

$$\theta = \pi \times |\vec{v}|$$

- *Rationale:* A max intensity emotion (Magnitude 1.0) represents a complete 180° turn away from neutrality.

Step 5: Quaternion Instantiation

The system constructs the quaternion using the Axis-Angle formulation:

$$q_{\text{current}} = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)(u_x i + u_y j + u_z k)$$

This results in the four components:

- $w = \cos(\theta/2)$
- $x = u_x \sin(\theta/2)$
- $y = u_y \sin(\theta/2)$
- $z = u_z \sin(\theta/2)$

This mapping ensures that a neutral state results in the identity quaternion ($1 + 0i + 0j + 0k$), while an intense emotion results in a quaternion representing a significant rotation away from neutral.

3.3 Calculating Emotional Work

A core metric of Project L.O.V.E. is "Emotional Work"—the quantification of the effort required to change states.

Transition Quaternion

To find the rotation that moves the user from a previous state (q_{start}) to a new state (q_{target}), we calculate the product of the target and the inverse (conjugate) of the start. $q_{\text{transition}} = q_{\text{target}} \times q_{\text{start}}^{-1}$ Since we use unit quaternions, the inverse is the conjugate ($q^* = w - xi - yj - zk$).

$$q_{\text{transition}} = q_{\text{target}} \times q_{\text{start}}^*$$

Angular Distance (ϕ)

The magnitude of the transition is given by the angle ϕ :

$$\phi = 2 \arccos(|w_{\text{transition}}|)$$

- **Important:** We use $|w_{\text{transition}}|$ (absolute value) to account for the "double cover" of quaternions (q and $-q$ represent the same rotation).
- **Interpretation:**
 - Small ϕ : The user is stable or ruminating.
 - Large ϕ (e.g., > 1.5 radians): The user is undergoing a radical shift (e.g., Despair to Hope).

3.4 Spherical Linear Interpolation (SLERP)

To visualize the transition in the Experience module, linear interpolation (LERP) is prohibited. LERP cuts through the sphere, shortening the radius (magnitude) and varying the angular

velocity.

The Versor must use SLERP:

$$\text{SLERP}(q_1, q_2, t) = \frac{\sin((1-t)\Omega)}{\sin(\Omega)}q_1 + \frac{\sin(t\Omega)}{\sin(\Omega)}q_2$$

where $\cos(\Omega) = q_1 \cdot q_2$ (the dot product).

SLERP ensures constant angular velocity. This is crucial for the haptic feedback system; a constant velocity allows for consistent vibration patterns that map to the "friction" of the turn.¹

4. Software Requirements: The Versor Module

4.1 Functional Requirements

FR-01: Input Sanitization and Validation

The system SHALL accept a JSON payload containing the current_vac vector.

The system SHALL validate that valence, arousal, and connection are float values between -1.0 and 1.0.

The system SHALL clamp values exceeding this range to the nearest boundary (e.g., 1.0001 -> 1.0) to handle floating-point drift from the Listener's LLM output.

FR-02: Zero-Vector Handling

The system SHALL explicitly check for a zero magnitude vector (\$\$).

IF the input is a zero vector:

The system SHALL return the Identity Quaternion \$\$.

The system SHALL skip axis normalization (to prevent DivisionByZero errors).

FR-03: VAC to Quaternion Conversion

The system SHALL implement the VAC_to_Quaternion algorithm defined in Section 3.2.

The system SHALL output quaternions in Scalar-First format (w, x, y, z) to maintain consistency with the theoretical documentation, despite some libraries defaulting to Scalar-Last.

FR-04: Transition Calculation

The system SHALL accept an optional previous_quaternion state.

IF previous_quaternion is provided:

The system SHALL calculate the transition_quaternion (q_{trans}).

The system SHALL calculate the angular_distance (ϕ) in radians.

IF previous_quaternion is NOT provided (start of session):

The system SHALL assume previous_quaternion is Identity.

The angular_distance will equal the magnitude of the current state.

FR-05: Path Generation (SLERP)

The system SHALL expose an endpoint to generate an interpolation path.

Input: start_quat, target_quat, steps (integer, default 60).

Output: An array of steps quaternions representing the geodesic arc.

Critical Logic: The system MUST implement the "Double Cover" check.

Calculate dot product $d = q_{\text{start}} \cdot q_{\text{target}}$.

IF $d < 0.0$:

Negate target_quat ($q_{\text{target}} = -q_{\text{target}}$).

(This ensures the path takes the shortest route, $\leq 180^\circ$, rather than the long route $> 180^\circ$).

FR-06: Velocity Analysis (Elasticity)

The system SHALL calculate the "Elasticity" (E) of the emotional shift.

$$E = \frac{\phi}{\Delta t}$$

where Δt is the time elapsed since the last input.

The system SHALL compare E against a configurable FLOODING_THRESHOLD.

IF $E > \text{FLOODING_THRESHOLD}$:

The system SHALL flag the response with `is_flooding = True`.

(This flag triggers specific haptic feedback in the Experience module to warn the user of overwhelm).

FR-07: Insight Generation (Axis Dominance)

The system SHALL analyze the transition_quaternion to identify the dominant axis of change.

Logic:

Extract vector part $\vec{v}_{\text{trans}} = [x_{\text{trans}}, y_{\text{trans}}, z_{\text{trans}}]$.

Find $\max(|x_{\text{trans}}|, |y_{\text{trans}}|, |z_{\text{trans}}|)$.

IF x is max: Return `InsightType.VALENCE_SHIFT`.

IF y is max: Return `InsightType.AROUSAL_SHIFT`.

IF z is max: Return `InsightType.CONNECTION_SHIFT`.

4.2 Non-Functional Requirements

NFR-01: Latency

The Versor Compute Loop (Input -> Math -> Path Generation -> Response) MUST complete in $< 50\text{ms}$ (P99) on standard serverless hardware (e.g., AWS Lambda 128MB or equivalent container).

Rationale: This ensures the visual feedback in the "Soul Sphere" feels instantaneous to the user's voice input.

NFR-02: Precision

The system MUST perform all internal calculations using 64-bit floating point numbers (float64) to minimize accumulation errors during conjugate multiplication and interpolation.

The system MAY downcast outputs to 32-bit floats (float32) for the API response payload to reduce bandwidth, as the visual rendering engine (React Three Fiber) typically operates in 32-bit precision.

NFR-03: Statelessness

The Versor module MUST be stateless. It should not maintain a persistent connection to the

database. All necessary state (previous quaternion) must be passed in the request payload or retrieved by the orchestrator (Observer) before calling the Versor.

NFR-04: Library Constraints (SciPy)

The system MUST utilize `scipy.spatial.transform` for robust rotation logic.

Constraint: The implementation MUST strictly control the quaternion component order.

Legacy SciPy versions used `scalar_last` (x, y, z, w) .

Newer SciPy versions ($\geq 1.4.0$) standardized on `scalar_last` but allow configuration.

Requirement: The Versor code MUST explicitly handle the mapping between Project L.O.V.E.'s theoretical `scalar_first` (w, x, y, z) and the library's expectation. See Section 6 for implementation details.⁴

5. API Interface Specifications

This section defines the data contracts for feeding into Claude or any other implementation agent.

5.1 Data Models (Pydantic)

Python

```
from pydantic import BaseModel, Field, validator
from typing import List, Optional
import math
```

```
class VACVector(BaseModel):
    """Normalized Valence-Arousal-Connection Vector"""
    valence: float = Field(..., ge=-1.0, le=1.0, description="Hedonic tone")
    arousal: float = Field(..., ge=-1.0, le=1.0, description="Energy level")
    connection: float = Field(..., ge=-1.0, le=1.0, description="Relational alignment")
```

```
class Quaternion(BaseModel):
    """Unit Quaternion in Scalar-First notation (w, x, y, z)"""
    w: float
    x: float
    y: float
    z: float

    @validator('w', 'x', 'y', 'z')
    def check_normalization(cls, v, values):
        # Note: Strict normalization check usually done on the whole object
        # This is a placeholder for the logic
        return v
```

```

class StateRequest(BaseModel):
    """Payload sent to Versor"""
    current_vac: VACVector
    previous_state: Optional[Quaternion] = None
    time_delta_seconds: float = Field(1.0, gt=0.0)

class TrajectoryResponse(BaseModel):
    """Output from Versor"""
    current_state: Quaternion
    transition_quaternion: Quaternion
    angular_distance_radians: float
    elasticity_metric: float
    is_flooding: bool
    insight_code: str # "VALENCE_SHIFT", "CONNECTION_SHIFT", etc.
    interpolation_path: List[Quaternion] # For animation frames

```

5.2 gRPC Service Definition (Proto3)

For high-performance internal communication, gRPC is recommended over REST.

Protocol Buffers

```

syntax = "proto3";

package versor;

service VersorEngine {
    // Main calculation endpoint
    rpc CalculateState (StateRequest) returns (TrajectoryResponse);
}

message Vector3 {
    double x = 1; // Valence
    double y = 2; // Arousal
    double z = 3; // Connection
}

message Quaternion {
    double w = 1;
    double x = 2;
    double y = 3;
}

```

```

double z = 4;
}

message StateRequest {
  Vector3 input_vac = 1;
  Quaternion previous_state = 2;
  double time_delta = 3;
}

message TrajectoryResponse {
  Quaternion target_state = 1;
  Quaternion transition_delta = 2;
  double work_radians = 3;
  double velocity = 4;
  bool is_flooding = 5;
  string dominant_axis = 6;
  repeated Quaternion path_frames = 7;
}

```

6. Implementation Strategy & Tech Stack

6.1 Python Environment & Dependencies

The Versor relies on the Python scientific stack.

- **Python Version:** 3.10+ (Required for structural pattern matching and type hinting).
- **Core Libraries:**
 - numpy: Essential for vector math.
 - scipy: Specifically `scipy.spatial.transform.Rotation` and `Slerp`.
 - fastapi: For the HTTP interface (if REST is used).
 - uvicorn: ASGI server.

Critical Note on SciPy Versioning:

Recent updates to scipy have introduced changes to quaternion handling.

- **Requirement:** Use `scipy >= 1.12.0`.
- **Implementation Detail:** As of SciPy 1.4+, `Rotation.from_quat` expects scalar-last (`x, y, z, w`). However, discussions in the SciPy community⁵ indicate support for `scalar_first` flags in newer versions.
- **Safe Implementation:** To ensure absolute correctness regardless of minor version flux, the Versor implementation should manually map inputs.

Pseudo-Code for Safe Rotation Creation:

Python

```

from scipy.spatial.transform import Rotation as R
import numpy as np

def create_rotation_from_love_quat(q_love):
    # Project LOVE uses (w, x, y, z)
    # SciPy uses (x, y, z, w) by default
    # We explicitly reorder to match SciPy's expectation
    sci_py_quat = [q_love.x, q_love.y, q_love.z, q_love.w]
    return R.from_quat(sci_py_quat)

def export_love_quat(r_object):
    # Convert back to (w, x, y, z) for L.O.V.E. standard
    q = r_object.as_quat() # returns (x, y, z, w)
    return Quaternion(w=q, x=q, y=q, z=q)

```

6.2 Database Integration (PostgreSQL + pgvector)

While the Versor is stateless, its outputs feed the Observer. The schema requirements for the Observer to support Versor data are:

- **Extension:** CREATE EXTENSION vector;
- **Column Type:** The quaternion state MUST be stored as vector(4).
- **Indexing:** The quaternion_state column requires an HNSW index for similarity search.
 - CREATE INDEX ON user_trajectory USING hnsw (quaternion_state vector_cosine_ops);
 - *Note on Distance Metric:* While "Cosine Distance" is standard for embeddings, for Unit Quaternions, the cosine distance ($1 - \cos(\theta)$) is monotonically related to the angular distance. Thus, cosine similarity search effectively finds the "nearest rotation".⁶

7. Testing, Validation, and QA

7.1 Canonical Transition Tests

The system MUST pass a suite of "Canonical Transitions"—pre-calculated shifts defined by the theoretical framework.

Test Case A: "Anger to Calm"

- **Input VAC (Anger):** $[-0.5, 0.8, -0.2]$ (Action-oriented, negative valence, slight disconnection).
- **Input VAC (Calm):** $[0.8, -0.6, 0.5]$ (Positive valence, low arousal, connected).
- **Expected Result:**
 - ϕ (Work) should be Large (> 1.5 rad).
 - Dominant Axis should likely be y (Arousal shift) or x (Valence shift).

Test Case B: "Pity to Compassion"

- **Input VAC (Pity):** $[-0.3, -0.2, -0.6]$ (Negative connection).
- **Input VAC (Compassion):** $[-0.3, -0.2, +0.8]$ (Positive connection).
- **Expected Result:**
 - ϕ should be substantial (crossing the hemisphere).
 - Dominant Axis MUST be z (Connection).
 - If the system reports this as a generic Valence shift, the Versor logic is failing to capture the project's core differentiator.

7.2 Property-Based Testing

Using the hypothesis library, the Versor logic should be subjected to randomized input vectors.

- **Invariant 1:** Output quaternions must always be Unit Quaternions (norm ≈ 1.0).
- **Invariant 2:** The SLERP path between a quaternion and itself must be length 0.
- **Invariant 3:** The SLERP path between q and $-q$ must be length 0 (Double Cover verification).

7.3 Performance Benchmarking

A performance test suite using locust or pytest-benchmark must be established.

- **Target:** 1,000 sequential calculations in < 1 second.
- **Resource Monitoring:** Ensure memory usage remains flat (no leaks in the NumPy array allocation).

8. Risk Management and Edge Cases

8.1 The "Connection" Ambiguity

Risk: While Valence and Arousal are well-documented (Russell's Circumplex), "Connection" is the novel variable. LLMs may struggle to extract this accurately.

Versor Mitigation: The Versor assumes the input is "Ground Truth." However, if the connection scalar oscillates rapidly (noise), the "Soul Sphere" will flicker efficiently.

Solution: Implement a Low-Pass Filter (Smoothing) in the Versor.

$$q_{\text{smoothed}} = \text{SLERP}(q_{\text{prev}}, q_{\text{new}}, \alpha)$$

where α is a smoothing factor (e.g., 0.1). This dampens high-frequency noise from the Listener module.

8.2 Singularity Management

Risk: Division by zero during axis normalization.

Mitigation: The explicit check for $|\vec{v}| < \epsilon$ (FR-02) is critical. The threshold ϵ should be set to $1e^{-6}$.

8.3 User Comprehension

Risk: Users will not understand what a "Quaternion" is.

Mitigation: The Versor provides the `insight_code` and `dominant_axis`. The Experience module/UI must translate this.

- Instead of "You rotated 0.5 radians on the Z-axis," the UI displays: "You are reconnecting with yourself."

9. Conclusion

The Versor module is the rigorous mathematical core of Project L.O.V.E. By translating the abstract, qualitative concepts of the "Atlas of the Heart" into the precise, non-commutative algebra of quaternions, it provides a robust framework for visualizing the trajectory of the human soul.

This specification outlines a system that is not merely a calculator, but a translation engine—converting the "Work" of emotional growth into a digital reality. Through the implementation of the VAC model, the resolution of Gimbal Lock via quaternions, and the precise application of SLERP for transition visualization, the Versor enables a new class of affective computing applications that honor the complexity, fluidity, and depth of the human experience. The roadmap provided here—from the theoretical topology to the specific Python implementation details—ensures that the resulting software will be scalable, accurate, and psychologically valid.

Works cited

1. L.O.V.E. Project Software Requirements.pdf
2. `scipy.spatial.transform._rotation.Slerp` - Document, accessed December 2, 2025, https://pydocs.github.io/p/scipy/1.8.0/api/scipy.spatial.transform._rotation.Slerp.html
3. `as_quat` — SciPy v1.16.2 Manual, accessed December 2, 2025, https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.transform.Rotation.as_quat.html
4. `from_quat` — SciPy v1.16.1 Manual, accessed December 2, 2025, https://docs.scipy.org/doc/scipy-1.16.1/reference/generated/scipy.spatial.transform.Rotation.from_quat.html
5. `dist` - Angular distance in radians - MATLAB - MathWorks, accessed December 2, 2025, <https://www.mathworks.com/help/nav/ref/quaternion.dist.html>